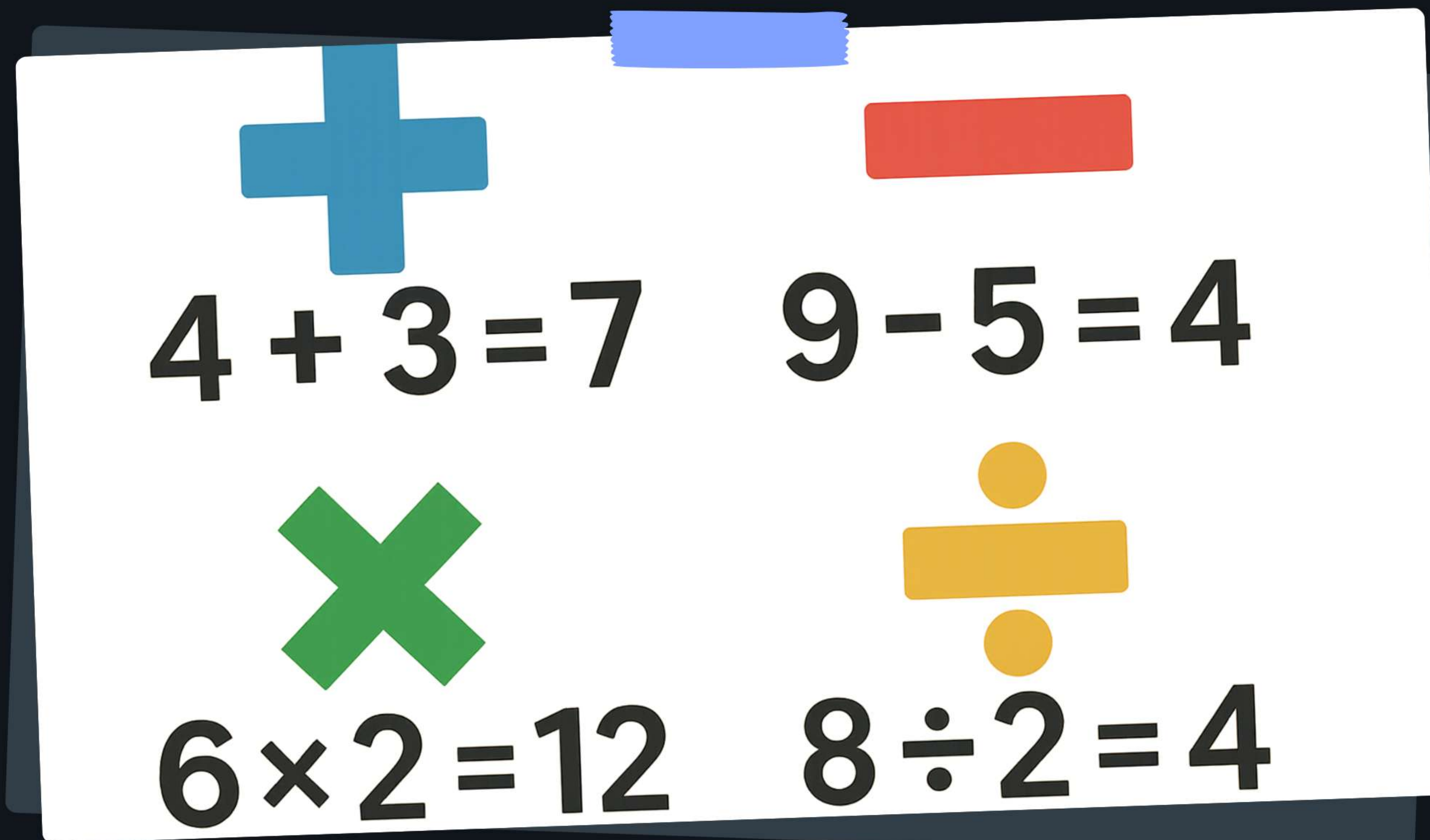


Java para Iniciantes: Tipos de Dados e Operadores com NetBeans

Dominando os Fundamentos da Programação em Java para o 10º Ano



The notepad displays four arithmetic operations arranged in a 2x2 grid:

- Top-left: A blue plus sign ($+$) above the equation $4 + 3 = 7$.
- Top-right: A red minus sign ($-$) above the equation $9 - 5 = 4$.
- Bottom-left: A green multiplication sign ($\times$) above the equation $6 \times 2 = 12$.
- Bottom-right: A yellow division sign ($\div$) above the equation $8 \div 2 = 4$.

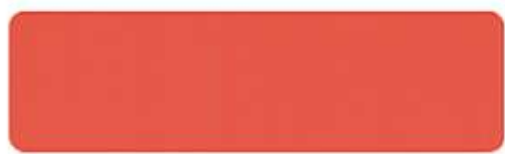
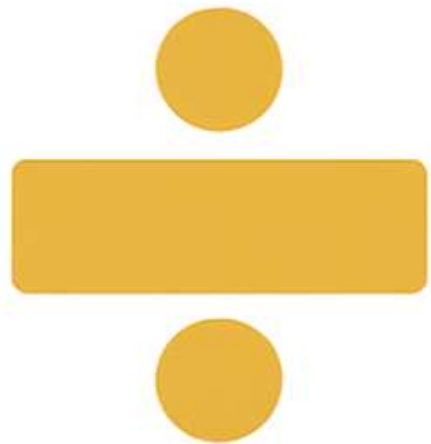


Sumário

- Introdução ao Java e NetBeans
- Tipos de Dados: Primitivos e String
- Operadores: Aritméticos, Relacionais, Lógicos
- Atribuição e Atribuição Composta
- Exemplos Práticos e Exercícios




$$4 + 3 = 7$$

$$6 \times 2 = 12$$

$$9 - 5 = 4$$

$$8 \div 2 = 4$$

Adição, subtração, multiplicação e divisão: operações com números naturais.

Bem-vindos ao Java!

Java é uma linguagem de programação robusta e amplamente utilizada, essencial para o desenvolvimento de sistemas complexos e aplicações modernas que permeiam nosso dia a dia. Nesta aula, exploraremos seus conceitos fundamentais, desde tipos de dados e operadores até exemplos práticos, para que você construa uma base sólida em programação.



$$\begin{array}{r} 4 + 3 \\ 7 \end{array}$$

$$\begin{array}{r} 9 - 5 \\ 4 \end{array}$$

$$\begin{array}{r} 6 \times 2 \\ 12 \end{array}$$

$$\begin{array}{r} 8 \div 4 \\ 2 \end{array}$$

Adição, subtração, multiplicação e divisão: operações aritméticas básicas.

Por Que NetBeans?

- IDE (Ambiente de Desenvolvimento Integrado): Simplifica a codificação.
- Interface intuitiva: Facilita o aprendizado.
- Autocompletar código: Agiliza a escrita.
- Depuração integrada: Ajuda a corrigir erros.



Seu Primeiro Projeto Java

1

Criar Novo Projeto

No NetBeans, selecione "File > New Project" e escolha "Java with Ant > Java Application".

2

Definir Classe Principal

Nomeie o projeto e a classe principal (*main class*), que é o ponto de partida do programa.

3

Escrever "Hello World!"

Dentro do método `main`, digite `System.out.println("Hello World!");` para exibir a mensagem.



Anatomia do 'Hello World!'

``public static void main(String[] args)`` é o ponto de entrada do programa.

``System.out.println()`` exibe texto no console. Comentários (linhas que começam com ``//`` ou blocos ``/ /``) explicam o código sem serem executados.



Console interativo: conversão de decimal para binário com botões e indicadores.



Introduzindo Variáveis

Variáveis são "contêineres" na memória do computador para armazenar dados. Cada variável precisa de um *tipo* (o que ela guarda) e um *nome* (como identificá-la). Isso permite organizar e manipular informações em seus programas Java.

A		B		C	
n	$6+n$	n	$4n$	n	$\sqrt{n}$
0	6	0	0	0	·000
1	7	1	4	1	1·000
2	8	2	8	2	1·414
3	9	3	12	3	1·732
.	.	.	.	.	.
.	.	.	.	.	.
.	.	.	.	.	.

Tabelas A, B e C: exemplos de séries numéricas simples.



7 > 3

5 = 5

4 < 9

Comparando valores: maior que, igual a, e menor que, em expressões.

Tipos Primitivos: O Básico

- Armazenam valores simples diretamente na memória.
- São fundamentais para operações básicas.
- Incluem: int, float, double, char, boolean.
- Ocupam espaço fixo e pré-definido.



`int`: Números Inteiros

O tipo de dado `int` armazena números inteiros (positivos, negativos e zero), com um alcance aproximado de -2 bilhões a +2 bilhões. É fundamental para contagens e identificadores.



`float`: Precisão Simples

O tipo `float` armazena números decimais com *precisão simples*, ou seja, menos casas decimais. É ideal para valores que não exigem alta exatidão, como preços. Para indicar um `float`, adicione 'f' ou 'F' ao final do número. Exemplo: `float preco = 19.99f`;

`double`: Precisão Dupla

O tipo `double` armazena números decimais com *precisão dupla*, significando mais casas decimais e maior exatidão. É a escolha padrão para cálculos científicos ou financeiros. Exemplo: `double pi = 3.14159`;



`char`: Caracteres Únicos

O tipo primitivo `char` armazena um único caractere Unicode (um sistema de codificação que permite representar todos os caracteres da maioria dos sistemas de escrita do mundo), como letras, números ou símbolos. Caracteres são sempre delimitados por aspas simples, por exemplo: `char inicial = 'J';` e `char simbolo = '#';`.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■
1	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	■
8	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■
9	■	'	'	■	■	■	■	■	■	■	■	■	■	■	■	■
A	¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	®	¯	°	±
B	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿	À	Á
C	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï	Ð	Ñ
D	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß	à	á
E	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï	ð	ñ
F	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ß		

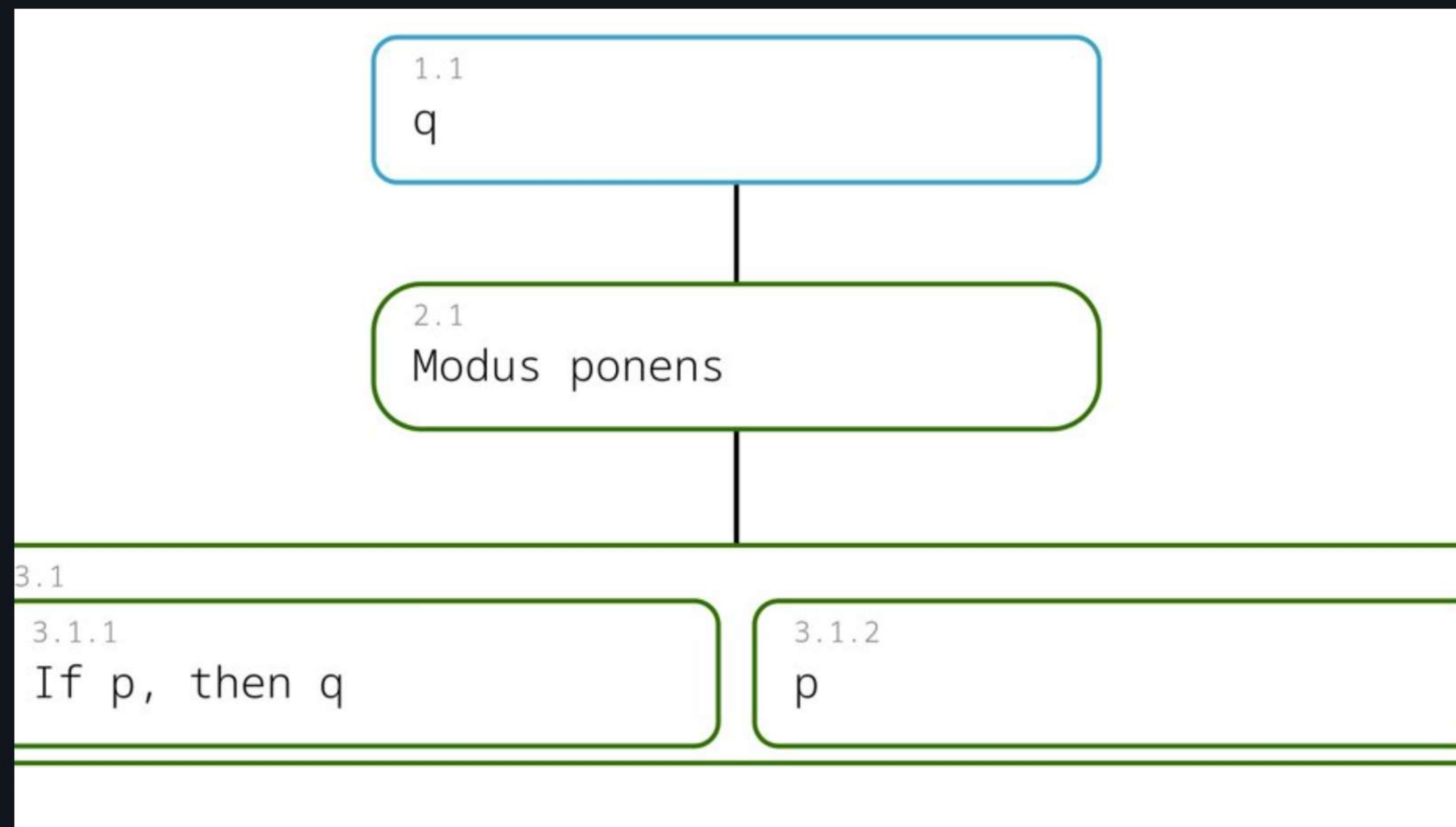
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	000	001	002	003	004	005	006	007	008	009	010	011	012	013	014	015
1	016	017	018	019	020	021	022	023	024	025	026	027	028	029	030	031
2	032	033	034	035	036	037	038	039	040	041	042	043	044	045	046	047
3	048	049	050	051	052	053	054	055	056	057	058	059	060	061	062	063
4	064	065	066	067	068	069	070	071	072	073	074	075	076	077	078	079
5	080	081	082	083	084	085	086	087	088	089	090	091	092	093	094	095
6	096	097	098	099	100	101	102	103	104	105	106	107	108	109	110	111
7	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
8	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143
9	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159
A	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175
B	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191
C	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207
D	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223
E	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239
F	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255

Tabela de codificação de caracteres ISO/IEC 8859-1 e mapeamento decimal.



`boolean`: Verdadeiro ou Falso

O tipo `boolean` armazena apenas `true` (verdadeiro) ou `false` (falso), essencial para o controle de fluxo (a ordem de execução das instruções) e a lógica condicional (decisões baseadas em condições).



Modus Ponens: Se 'p', então 'q'; 'p' implica na conclusão 'q'.

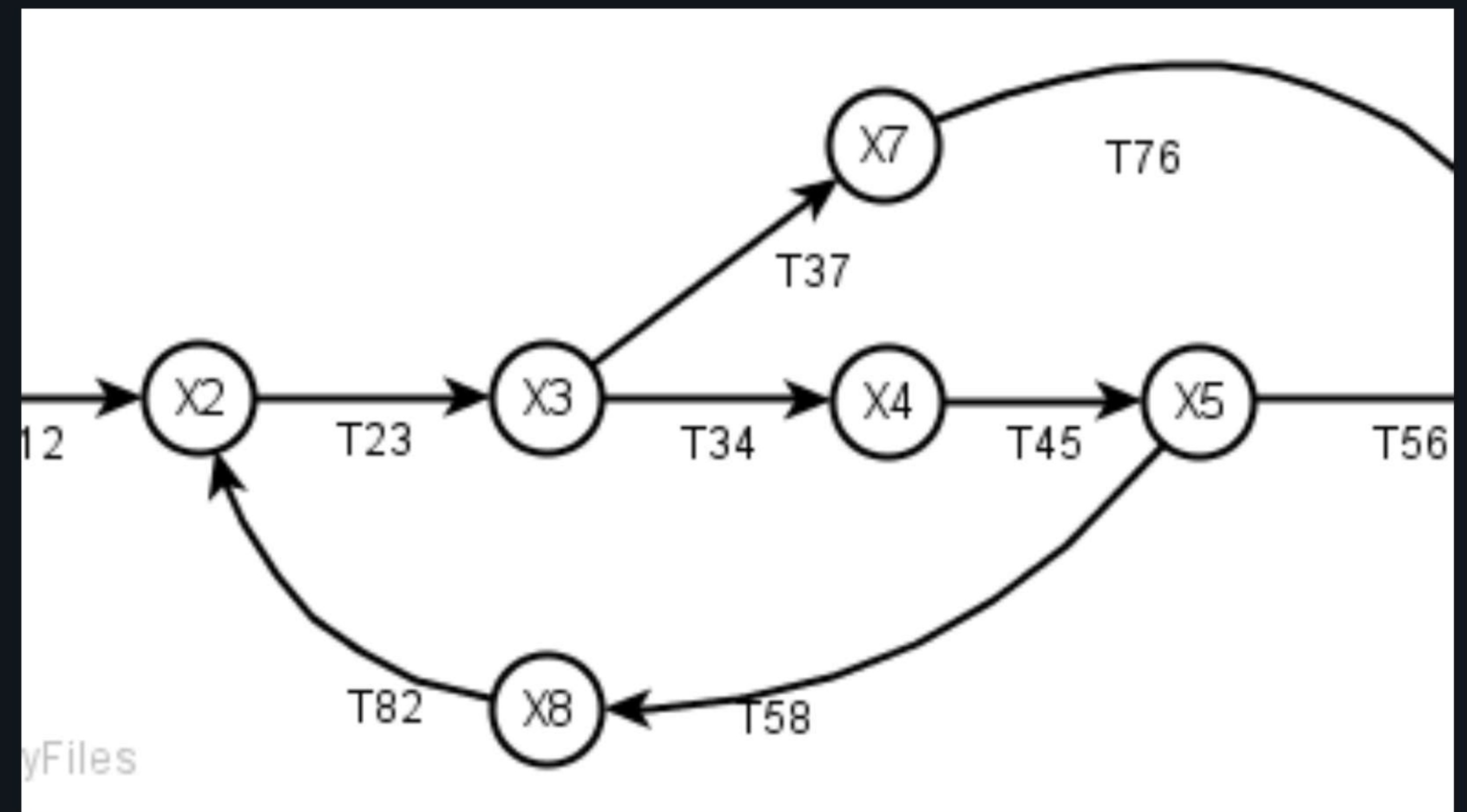
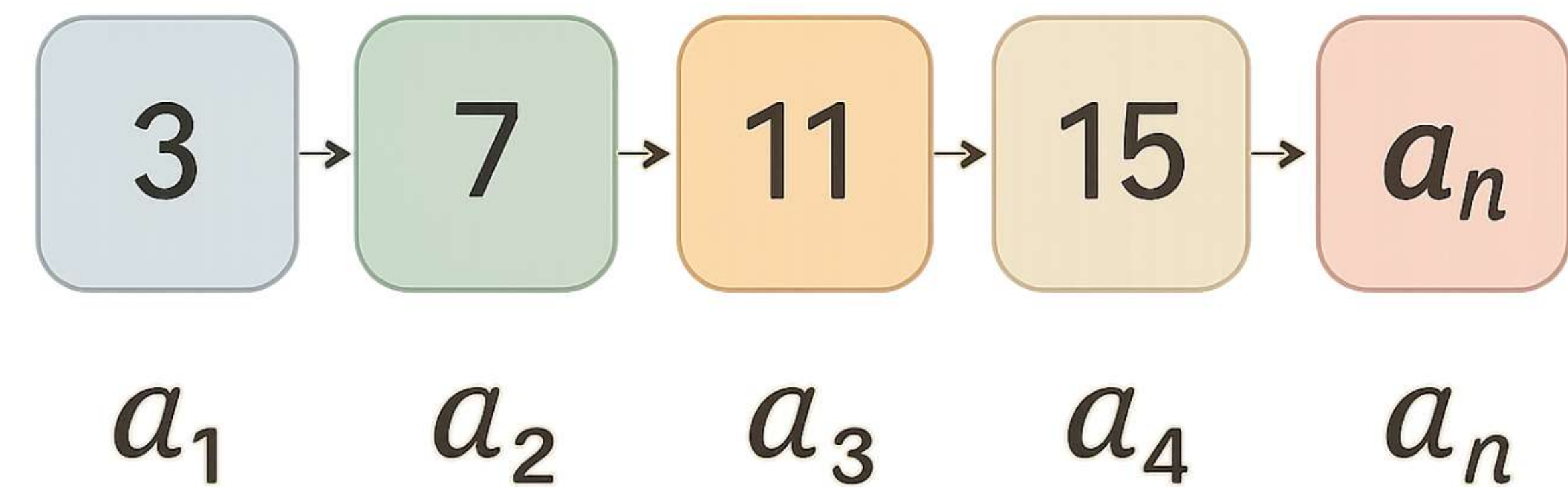


Diagrama de fluxo de sinal: representação de rede com loops e ramificações.



Além dos Primitivos: `String`

Apresentamos o tipo `String`, uma sequência de caracteres (texto). Diferente dos tipos primitivos, `String` é uma **classe** (um objeto), oferecendo funcionalidades complexas. Declaramos como `String nome = "Maria";`, um exemplo de como armazenar dados textuais em Java.

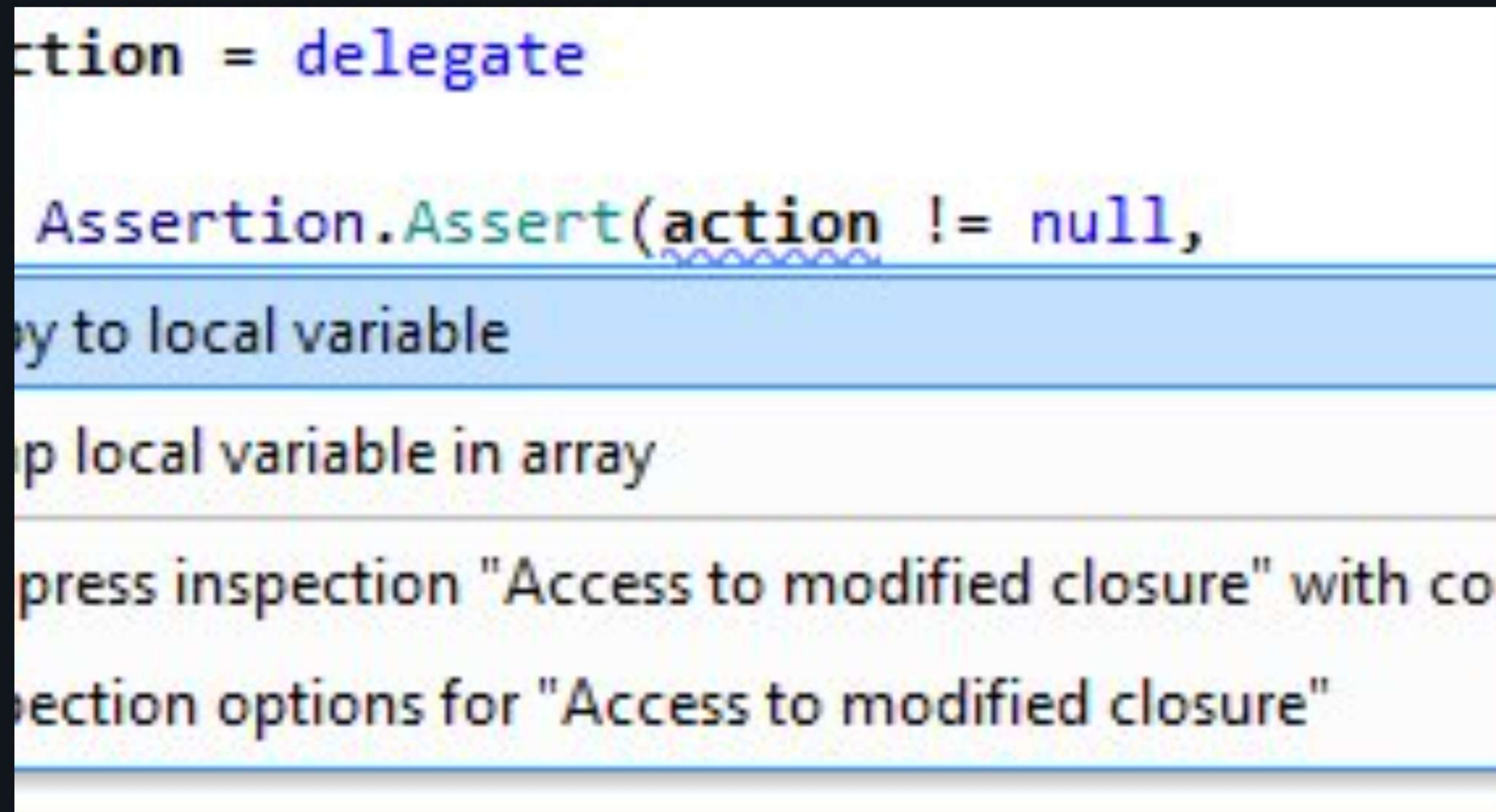


Sequência aritmética: 3, 7, 11, 15. Diferença constante de 4.

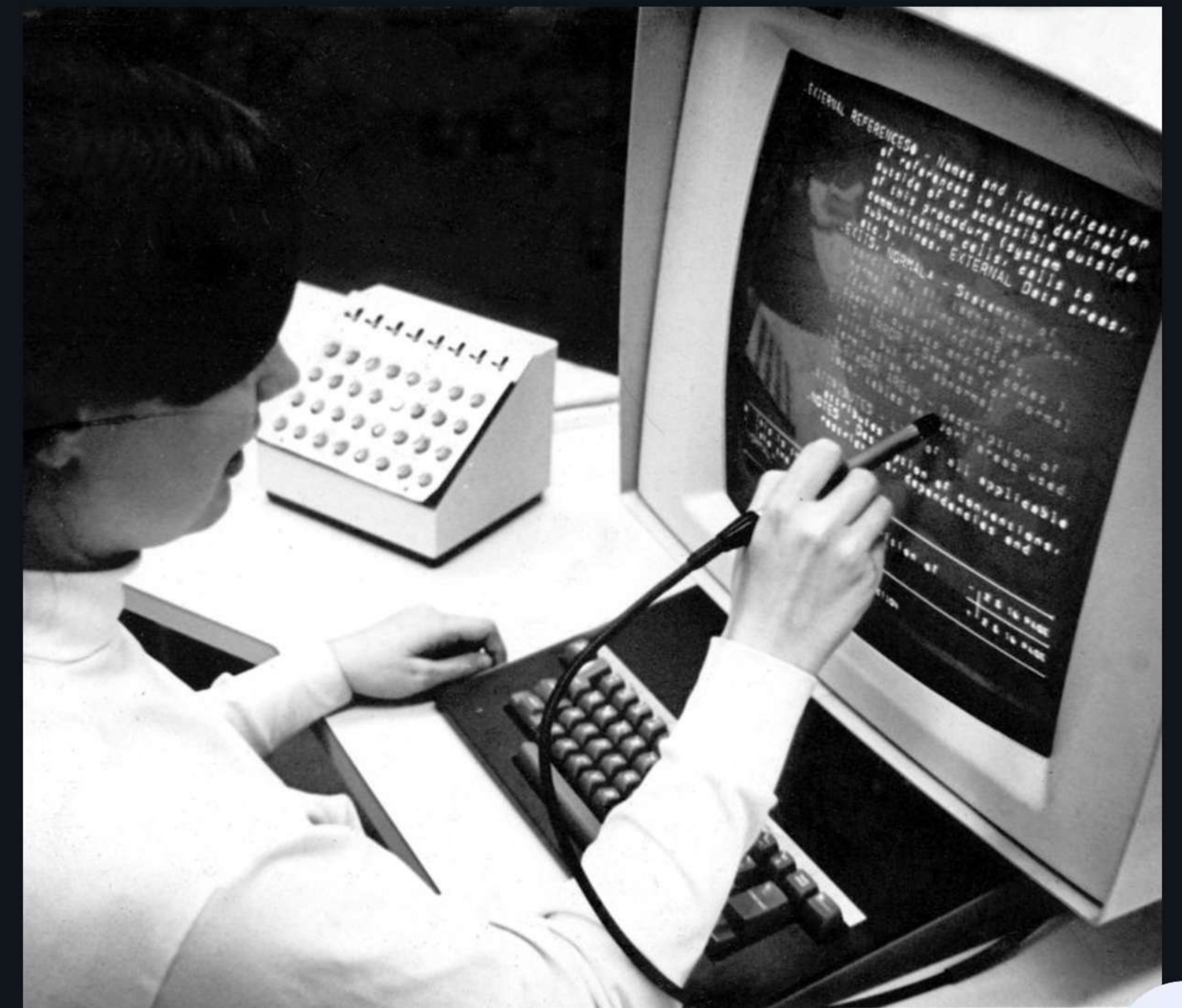


Manipulando `String`

Como um objeto, `String` permite operações como concatenação (unir textos com `+`) e inicialização com aspas duplas. Veja exemplos práticos de manipulação de texto.



ReSharper sugere melhorias no código C#, com foco em 'action'.



Usuário edita texto em terminal antigo, possivelmente programação.



Operadores: Ferramentas de Manipulação

Operadores Aritméticos

Realizam cálculos matemáticos básicos como soma, subtração, multiplicação e divisão. Essenciais para manipular valores numéricos em expressões.

Operadores Relacionais

Comparam dois valores, resultando em um booleano (verdadeiro ou falso). São fundamentais para tomadas de decisão na programação.

Operadores Lógicos

Combinam ou negam expressões booleanas, como "E" (AND), "OU" (OR) e "NÃO" (NOT). Cruciais para construir condições complexas.



$$\frac{1}{2} + \frac{3}{8}$$

$$\frac{4}{1} - \frac{1}{2}$$

$$\frac{3}{5} \times \frac{1}{3}$$

$$\frac{5}{2} \div \frac{7}{2}$$

Adição, subtração, multiplicação e divisão de frações e números mistos.

Operadores Aritméticos

- Adição (+), Subtração (-), Multiplicação (*) para cálculos.
- Divisão (/) inteira trunca, não arredonda resultados.
- Módulo (%) retorna o resto da divisão.
- Use módulo para verificar números pares/ímpares.



Precedência Aritmética

A precedência aritmética define a ordem de avaliação de operadores em expressões. Java segue o padrão PEMDAS (Parênteses, Expoentes, Multiplicação/Divisão, Adição/Subtração). O uso de parênteses altera essa ordem, garantindo o resultado desejado.



Incremento (++)

O operador de incremento (`++`) adiciona 1 ao valor de uma variável numérica. Na forma prefixada (`++x`), a variável é incrementada *antes* de ser usada na expressão. Já na forma posfixada (`x++`), a variável é usada na expressão *e depois* incrementada. Isso é crucial para o resultado final em operações complexas.

Decremento (--) e Exemplos

O operador de decremento (`--`) subtrai 1 do valor de uma variável numérica. Similarmente, temos a forma prefixada (`--x`) e a posfixada (`x--`). A diferença na ordem de execução da operação e do uso do valor na expressão permanece a mesma. Por exemplo, se `int a = 5; int b = a++;` então `b` será 5, e `a` será 6.



$$\begin{array}{r} 27 \\ + 35 \\ \hline \end{array}$$

$$\begin{array}{r} 146 \\ + 728 \\ \hline \end{array}$$

$$\begin{array}{r} 52 \\ - 19 \\ \hline \end{array}$$

$$\begin{array}{r} 804 \\ - 657 \\ \hline \end{array}$$

Problemas de adição e subtração: prática de operações aritméticas básicas.

Operadores Relacionais

- Comparam dois valores
- Retornam `true` ou `false`
- Essenciais para decisões no código
- Exemplos: `==`, `!=`, `>`, `<`, `>=`, `<=`



Operadores Lógicos

&& (E Lógico)

O operador `&&` retorna *true* (verdadeiro) apenas se AMBAS as expressões booleanas (que são valores que podem ser verdadeiros ou falsos) forem verdadeiras. É útil para condições que exigem múltiplos critérios.

|| (OU Lógico)

O operador `||` retorna *true* se PELA MENOS UMA das expressões booleanas for verdadeira. Ele é usado quando qualquer uma de várias condições é suficiente.

! (NÃO Lógico)

O operador `!` inverte o valor de uma expressão booleana, transformando *true* em *false* e vice-versa. É usado para negar uma condição existente.



Operador de Atribuição `=`

O operador de atribuição simples (`=`) é fundamental para variáveis, pois ele define o valor de uma variável. Por exemplo, em `int x = 10;`, o valor `10` é *atribuído* à variável `x`, não sendo uma comparação.



$$7 + \square = 10$$

$$12 - 5 = \square$$

$$4 \times \square = 20$$

$$\square : 3 = 9$$

Encontre os valores ausentes nas equações algébricas usando operações básicas.

Operadores de Atribuição Composta

- Atalhos para combinar operação e atribuição.
- Exemplos: ``+=``, ``-=``, ``*=``, ``/=``, ``%=``.
- ``x += 5;`` é o mesmo que ``x = x + 5;``.
- Otimizam o código, tornando-o mais legível.



Conversão Implícita (Widening)

Ocorre automaticamente quando um tipo de dado menor é atribuído a um tipo maior, sem perda de informação. Por exemplo, converter um `int` (inteiro) para um `double` (número de ponto flutuante) é implícito. O Java gerencia essa conversão de forma segura. Não há necessidade de um operador especial.

Conversão Explícita (Narrowing)

Requer o uso do operador de *cast* `(tipo)` para converter um tipo maior para um menor. Por exemplo, de `double` para `int`. Pode haver perda de dados, como a parte decimal de um número. Use com cautela e consciência dosagem.



Exemplo Prático: Calculadora Simples

Este exemplo de calculadora simples em Java demonstra a aplicação de variáveis, tipos de dados primitivos (como `int` e `double`), `String` (para manipulação de texto) para entrada/saída, e operadores aritméticos. Observe a interação e a lógica de um programa básico.



Cálculos financeiros: calculadora, gráficos e análise de dados.



Exemplo Prático: Lógica de Idade

Este exemplo demonstra como operadores relacionais (como $\geq$) e lógicos (como `&&`, que significa 'E') são usados para verificar a maioridade. Combinamos condições para tomar decisões complexas, fundamental para validação de dados.



Revisão Essencial

Revisitamos os tipos de dados primitivos, como **int** (para números inteiros) e **boolean** (para valores lógicos verdadeiro/falso), e o tipo **String** (para sequências de caracteres). Compreendemos a função dos operadores aritméticos, relacionais (para comparações), lógicos (como AND e OR), de atribuição e compostos. A escolha correta do tipo e do operador é crucial para a lógica do programa.

Próximos Passos em Java

Com esta base sólida, avançaremos para o estudo das **estruturas de controle de fluxo** (que determinam a ordem de execução do código). Em seguida, exploraremos as **estruturas de repetição** (que permitem executar blocos de código múltiplas vezes). Estes conceitos são fundamentais para criar programas mais dinâmicos e complexos.



$$\begin{array}{r} 34 \\ + 58 \\ \hline 92 \\ \\ 81 \\ - 46 \\ \hline 35 \end{array}$$

$$\begin{array}{r} 47 \\ + 26 \\ \hline 73 \\ \\ 63 \\ - 29 \\ \hline 34 \end{array}$$

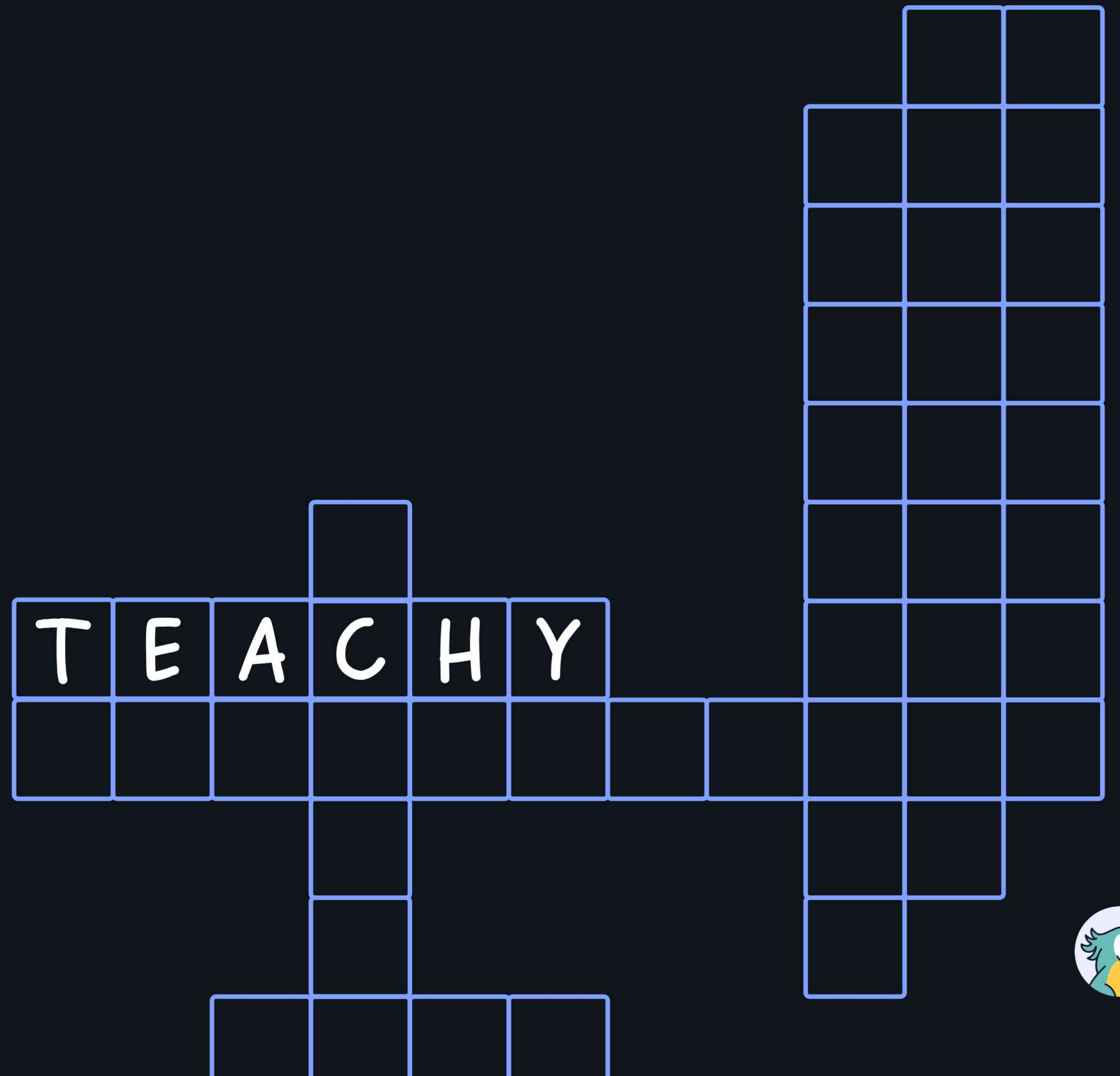
Adição e subtração de números naturais menores que 100.

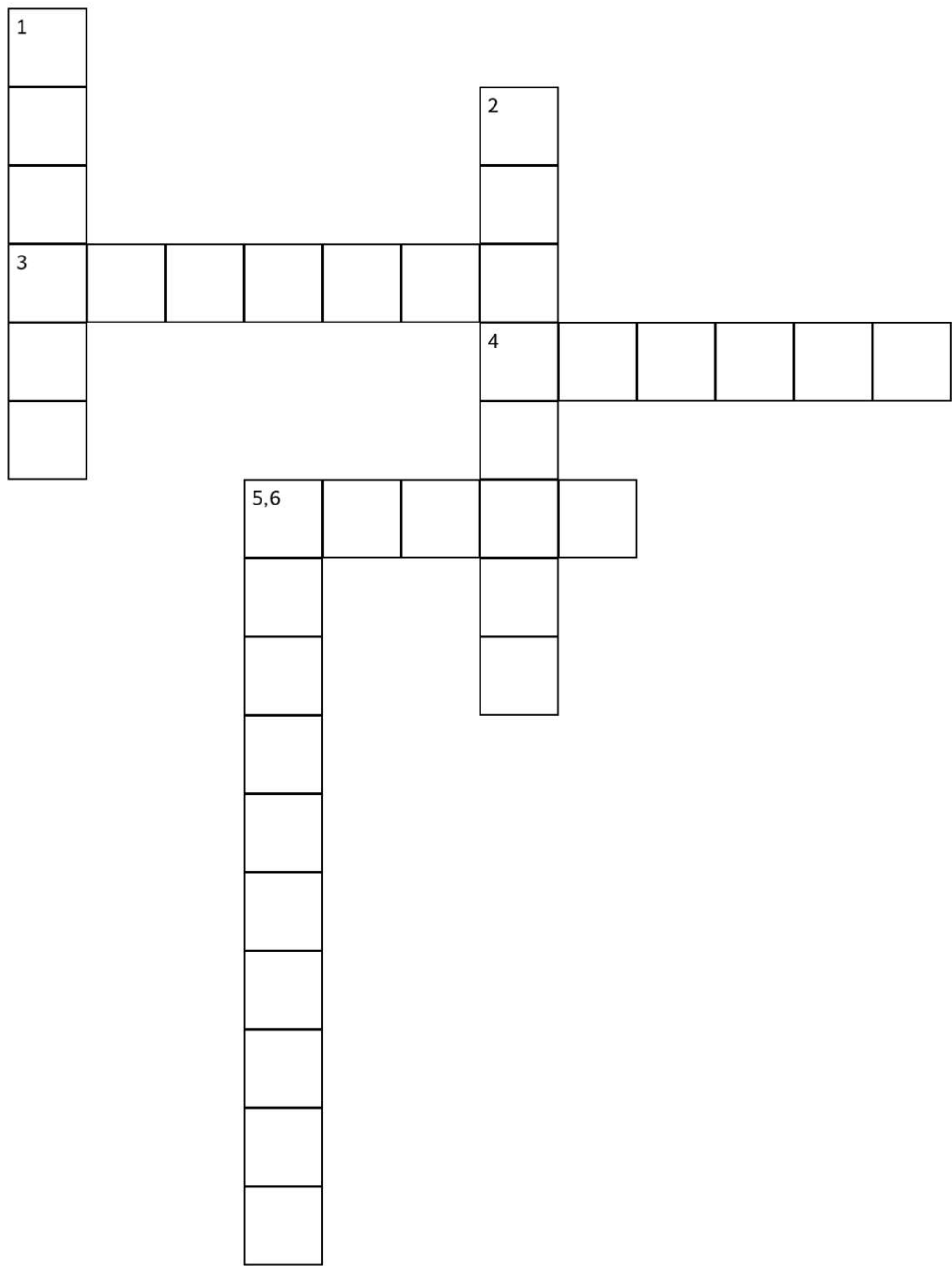
Desafios para Fixar!

- Declare variáveis de diferentes tipos e exiba-as.
- Calcule a média de notas usando operadores.
- Verifique se um número é par ou ímpar.
- Crie um programa de conversão de temperatura.



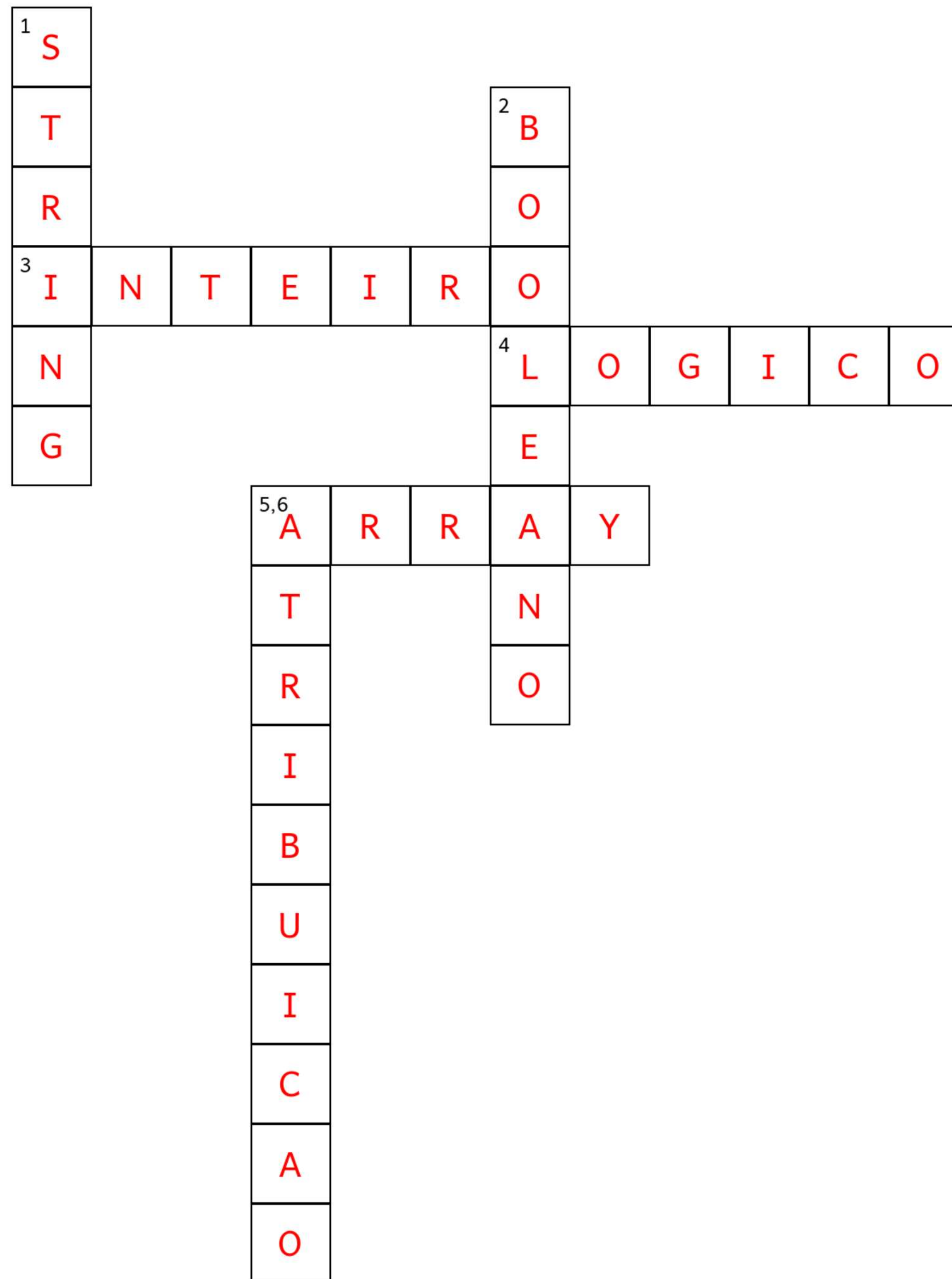
Cruzadinha





HORIZONTAL	VERTICAL
3. Tipo de dado primitivo em Java para números sem casas decimais.	1. Tipo de dado não primitivo em Java usado para armazenar sequências de caracteres.
4. Operador que combina expressões relacionais e resulta em verdadeiro ou falso.	2. Tipo de dado primitivo que representa um valor verdadeiro ou falso.
5. Estrutura de dados não primitiva que armazena uma coleção de elementos do mesmo tipo.	6. Operador em Java que serve para dar um valor a uma variável.





Conclusão

- Java usa tipos primitivos (int, float, char, boolean) e não primitivos (String) para armazenar dados.
- Operadores aritméticos (+, -, *, /, %) realizam cálculos essenciais em expressões numéricas.
- Operadores relacionais (==, !=, >, <) e lógicos (&&, ||, !) controlam o fluxo de decisões.
- NetBeans facilita a prática, declaração de variáveis e aplicação de operadores em código Java.

